



MODUL PRAKTIKUM

Sistem Operasi

IF25-12007 | Genap 2025/2026

Mata Kuliah Sistem Operasi (IF25-12007)

Semester Genap 2025/2026

Institusi Institut Teknologi Sumatera

TIM PENGAJAR & ASISTEN

Dosen Pengampu

Martin Clinton Tosima Manullang, S.T., M.T., Ph.D.

Ilham Firman Ashari, S.Kom., M.T

I Wayan Wiprayoga Wisesa, S.Kom., M.Kom.

Eko Dwi Nugroho, S.Kom., M.Cs.

Asisten Praktikum

Ahmat Prayoga Sembiring	123140053
Andryano Shevchenko Limbong	123140205
Aziz Kurniawan	122140097
Dito Rifki Irawan	122140153
Ebentua Philippus Limbong	123140086
Elsa Elisa Yohana Sianturi	122140135
Fathan Andi Kartagama	122140055
Miftahul Khoiriyah	123140064
Muhammad Bintang Al-Fasya	123140098
Muhammad Fadhil Zurani	122140146
Reza Chairul Manam	120140086
Stevanus Cahya Anggara	123140038
Varasina Farmadani	123140107
Zakhi Algifari	122140198

Contents

1	Penjadwalan Proses Round-Robin	5
1.1	Tujuan dan Output Praktikum.	5
1.1.1	Tujuan Praktikum	5
1.1.2	Output Praktikum	5
1.2	Dasar Teori	7
1.2.1	CPU Scheduling	7
1.2.2	Algoritma Round-Robin (FIFO)	8
1.2.3	Cooperative Scheduling vs Preemptive Scheduling	9
1.2.4	Field Priority pada Struct Thread.	9
1.2.5	Keterbatasan Round-Robin	10
1.3	Alat dan Bahan	11
1.3.1	Perangkat Keras	11
1.3.2	Perangkat Lunak (Bahan Praktikum)	11
1.4	Langkah-Langkah Praktikum.	13
1.4.1	Persiapan Lingkungan	13
1.4.2	Sesi 1: Menjalankan Demo Round-Robin dengan Thread Berprioritas Berbeda	15
2	Penjadwalan Proses Berbasis Prioritas	19
2.1	Tujuan dan Output Praktikum.	19
2.1.1	Tujuan Praktikum	19
2.1.2	Output Praktikum	19
2.2	Dasar Teori	21
2.2.1	Priority Scheduling	21
2.2.2	Perbedaan Mendasar dengan Round-Robin	22
2.2.3	Starvation (Kelaparan)	22
2.2.4	Solusi: Aging	23

2.3	Alat dan Bahan	25
2.3.1	Perangkat Keras	25
2.3.2	Perangkat Lunak (Bahan Praktikum)	25
2.4	Langkah-Langkah Praktikum.	26
2.4.1	Persiapan Lingkungan	26
2.4.2	Sesi 1: Analisis Kode Scheduler Round-Robin yang Akan Dimodifikasi.	26
2.4.3	Sesi 2: Implementasi Priority Scheduling	28
2.4.4	Sesi 3: Kompilasi dan Verifikasi Sistem OS	30

Bibliography

33

CHAPTER 1

Penjadwalan Proses Round-Robin

1.1 Tujuan dan Output Praktikum

1.1.1 Tujuan Praktikum

Setelah menyelesaikan praktikum ini, mahasiswa diharapkan mampu:

1. Memahami konsep *CPU Scheduling* dan peran komponen *scheduler* dalam mengelola pembagian giliran waktu CPU antar *thread* pada tingkat kernel.
2. Menganalisis perilaku algoritma penjadwalan *Round-Robin* (FIFO) yang telah diimplementasikan pada Modul 8, khususnya pada fungsi `next_thread_to_run()` di `src/thread.c`.
3. Memahami konsep *time slice*, *cooperative scheduling*, dan implikasinya terhadap keadilan (*fairness*) pembagian CPU.
4. Menghubungkan perilaku *scheduler* Round-Robin dengan keluaran (*output*) sistem di QEMU dan membuktikan bahwa algoritma ini mengabaikan nilai prioritas *thread*.

1.1.2 Output Praktikum

Pada akhir praktikum ini, mahasiswa diharapkan menghasilkan:

- Analisis kritis terhadap algoritma *Round-Robin* yang berjalan di fungsi `next_thread_to_run()`.

- Tangkapan layar (*screenshot*) keluaran QEMU yang menunjukkan tiga *thread* dengan prioritas berbeda (namun tetap berjalan bergantian secara merata) sebagai bukti bahwa *Round-Robin* mengabaikan nilai prioritas.
- Laporan yang menjabarkan kelebihan dan keterbatasan *Round-Robin*, khususnya ketidakmampuannya mendahulukan *thread* berprioritas tinggi, sebagai landasan untuk modifikasi ke *Priority Scheduling* pada Modul 12.

1.2 Dasar Teori

1.2.1 CPU Scheduling

CPU Scheduling adalah mekanisme fundamental yang digunakan sistem operasi untuk memutuskan *thread* mana dari antrean *ready* yang akan mendapat giliran eksekusi di CPU pada suatu waktu tertentu [1]. Komponen yang menjalankan keputusan ini disebut *scheduler*. Tanpa *scheduler*, CPU hanya mampu menjalankan satu alur eksekusi tanpa kemampuan beralih ke tugas lain.

Analoginya seperti sebuah lapangan sepak bola yang hanya memiliki satu bola (CPU). Meski ada 11 pemain (*thread*) di lapangan, pada satu waktu tertentu hanya satu pemain yang bisa menguasai bola. Pelatih (*scheduler*) bertugas menentukan siapa yang harus menerima bola selanjutnya berdasarkan strategi tertentu.

Tujuan utama *CPU scheduling* meliputi:

- **Utilisasi CPU:** Memastikan CPU tidak pernah menganggur (*idle*) selama masih ada *thread* yang siap dijalankan. CPU yang menganggur sama dengan pemborosan sumber daya.
- **Fairness:** Memberikan kesempatan yang adil bagi setiap *thread* untuk menggunakan CPU, sehingga tidak ada *thread* yang terabaikan sepenuhnya.
- **Throughput:** Memaksimalkan jumlah *thread* yang berhasil menyelesaikan pekerjaannya per satuan waktu.
- **Response Time:** Meminimalkan waktu tunggu antara permintaan *thread* untuk dijalankan dan saat ia benar-benar mendapat giliran CPU.

Untuk mencapai tujuan-tujuan tersebut, sistem operasi menggunakan berbagai algoritma penjadwalan. Salah satu algoritma yang paling mendasar adalah *Round-Robin*, yang akan kita analisis pada modul ini.

1.2.2 Algoritma Round-Robin (FIFO)

Round-Robin (RR) adalah algoritma penjadwalan yang paling sederhana dan paling intuitif. Nama "Round-Robin" berasal dari istilah yang digunakan dalam turnamen olahraga, di mana setiap peserta mendapat giliran bermain secara berputar.

Prinsipnya serupa dengan antrean di loket bank: siapa yang datang lebih dulu, ia yang dilayani lebih dulu (*First-In, First-Out* / FIFO). Setelah mendapat giliran, *thread* berjalan hingga salah satu dari dua hal terjadi:

1. *Thread* secara sukarela melepas CPU (melalui `thread_yield()`).
2. *Thread* dihentikan paksa oleh *timer interrupt* setelah *time slice* habis (pada sistem *preemptive*).

Setelah melepas CPU, *thread* tersebut masuk kembali ke belakang antrean, menunggu giliran berikutnya. Proses ini berulang terus sehingga setiap *thread* mendapat giliran secara merata.

Implementasi Ready Queue sebagai Circular Array

Agar prinsip FIFO dapat berjalan efisien, *starter code* OS menggunakan *circular array* sebagai struktur data antrean *ready*. Dengan pendekatan ini, operasi penambahan (*enqueue*) dan pengambilan (*dequeue*) *thread* tidak perlu menggeser seluruh elemen array.

```
1 static struct thread *ready_queue[MAX_THREADS * 2]; /* 16 slot */
2 static uint32_t ready_head = 0; /* Indeks depan antrean */
3 static uint32_t ready_tail = 0; /* Indeks belakang antrean */
4 static uint32_t ready_count = 0; /* Jumlah thread di antrean */
```

Kode 1.1: Variabel antrean thread pada thread.c

Mekanisme *circular array* bekerja sebagai berikut:

- **Enqueue** (masukkan): Thread baru ditempatkan di posisi `ready_tail`, lalu `ready_tail` digeser maju satu posisi secara *modular* (kembali ke 0 jika sudah mencapai ujung array).

- **Dequeue** (keluarkan): Thread diambil dari posisi `ready_head`, lalu `ready_head` digeser maju satu posisi.

Berikut ilustrasi keadaan antrean saat tiga *thread* (A, B, C) sedang terdaftar dan *thread* A baru saja dikeluarkan untuk dieksekusi:

1	Indeks:	[0]	[1]	[2]	[3]	[4]	...
2	Isi:	-	B	C	-	-	
3			^		^		
4		ready_head		ready_tail			
5		(dequeue)		(enqueue)			

Kode 1.2: Ilustrasi circular array `ready_queue`

1.2.3 Cooperative Scheduling vs Preemptive Scheduling

Mekanisme antrean *Round-Robin* baru akan berfungsi jika ada proses yang mengatur perpindahan giliran CPU. Dalam praktiknya, perpindahan ini dapat terjadi melalui dua model:

1. **Cooperative Scheduling:** *Thread* secara sadar dan sukarela melepas CPU dengan memanggil fungsi seperti `thread_yield()`. Jika sebuah *thread* "egois" dan tidak pernah memanggil `yield()`, ia akan menguasai CPU selamanya, menyebabkan seluruh *thread* lain membeku. Model inilah yang digunakan oleh IF-OS saat ini.
2. **Preemptive Scheduling:** Sistem operasi menggunakan *timer interrupt* dari perangkat keras untuk secara paksa merebut CPU dari *thread* yang sedang berjalan setelah *time slice* tertentu (misalnya 10 milidetik). Model ini digunakan oleh semua OS modern (Windows, Linux, macOS).

1.2.4 Field Priority pada Struct Thread

IF-OS yang kita gunakan saat ini menerapkan *cooperative scheduling*, namun setiap *thread* sebenarnya sudah memiliki atribut prioritas. Atribut ini didefinisikan pada `src/thread.h` melalui field `priority` di dalam struct `thread`:

```
1 #define PRI_MIN      0      /* Prioritas terendah */
2 #define PRI_DEFAULT  31     /* Prioritas default */
3 #define PRI_MAX      63     /* Prioritas tertinggi */
```

Kode 1.3: Definisi konstanta prioritas pada thread.h

Field ini sudah diinisialisasi oleh `init_thread()` setiap kali *thread* dibuat. Namun, fungsi `next_thread_to_run()` pada implementasi *Round-Robin* **sama sekali tidak membaca field ini**. Akibatnya, *thread* berprioritas 10 dan *thread* berprioritas 50 diperlakukan setara, keduanya mendapat giliran berdasarkan urutan kedatangan saja. Keterbatasan inilah yang akan diatasi pada Modul 12.

1.2.5 Keterbatasan Round-Robin

Meskipun *Round-Robin* memberikan perlakuan yang adil bagi setiap *thread*, pendekatan ini memiliki beberapa keterbatasan:

- **Tidak ada diferensiasi kepentingan:** Sebuah *thread* yang menangani tugas kritis (misalnya penanganan *interrupt* atau pemrosesan I/O) mendapat giliran yang sama dengan *thread* yang hanya menjalankan tugas ringan.
- **Response time tidak optimal:** *Thread* penting yang baru masuk antrean harus menunggu semua *thread* di depannya selesai mendapat giliran, meskipun tugasnya lebih mendesak.
- **Tidak cocok untuk sistem real-time:** Sistem yang membutuhkan jaminan waktu respon (seperti kendali motor atau perangkat medis) tidak dapat mengandalkan Round-Robin.

Tabel 1.1 merangkum konsep-konsep penting pada modul ini.

Tabel 1.1: Tabel Ringkasan Dasar Teori Modul 11

Konsep	Penjelasan Singkat
CPU Scheduling	Mekanisme OS untuk memutuskan <i>thread</i> mana yang mendapat giliran CPU berikutnya.
Round - Robin (FIFO)	Algoritma penjadwalan berdasarkan urutan kedatangan; setiap <i>thread</i> mendapat giliran secara merata tanpa memperhatikan prioritas.
Time Slice / Quantum	Jatah durasi waktu CPU per giliran sebelum <i>thread</i> harus menyerahkan kendali.
Cooperative Scheduling	Model penjadwalan di mana <i>thread</i> secara sukarela melepas CPU melalui pemanggilan <code>yield()</code> .
Preemptive Scheduling	Model penjadwalan di mana OS memaksa perpindahan <i>thread</i> melalui <i>timer interrupt</i> .
Ready Queue (Circular Array)	Antrean penampung <i>thread</i> berstatus <code>THREAD_READY</code> ; dikelola dengan indeks <code>ready_head</code> dan <code>ready_tail</code> .

1.3 Alat dan Bahan

1.3.1 Perangkat Keras

- PC atau Laptop dengan arsitektur prosesor basis *host* x86/x64 yang mendukung kelanjutan kompilasi emulator.
- Memori minimal 4GB RAM ruang eksekusi.

1.3.2 Perangkat Lunak (Bahan Praktikum)

- OS *Host* utama (Windows dengan WSL/GitBash, Linux, atau macOS).
- Text Editor (misalnya Visual Studio Code) untuk menyunting berkas kode bahasa C dan *Assembly*.
- Instalasi Docker Engine (`docker.io`) untuk menyalakan lingkungan isolasi kompilasi.
- Emulator Hardware x86: `qemu-system-x86` (atau instalasi QEMU setara) yang akan mengeksekusi binari hasil OS.
- Direktori *Starter Code* OS 32-bit dengan implementasi `thread_create()`,

`thread_yield()`, dan `next_thread_to_run()` yang sudah berfungsi dari Modul 8 (berisi direktori `src` dengan kerangka lengkap `kernel.c`, `thread.h`, `thread.c`, `switch.asm`, `dsb`).

1.4 Langkah-Langkah Praktikum

Pada bagian ini, kita akan menguji dan menganalisis perilaku algoritma *Round-Robin* yang sudah diimplementasikan sejak Modul 8. Melalui pengamatan langsung di QEMU, kita akan membuktikan bahwa *scheduler* ini mengabaikan nilai prioritas *thread* dan memperlakukan semua *thread* secara setara.

1.4.1 Persiapan Lingkungan

Sebelum memulai praktikum, pastikan *toolchain* dan *source code* OS sudah tersedia. Karena praktikum ini merupakan lanjutan dari implementasi *multi-threading* (Modul 8) dan sinkronisasi (Modul 9–10), implementasi `thread_create()`, `thread_yield()`, dan `next_thread_to_run()` di `thread.c` **harus sudah selesai dan berjalan dengan benar**. Jika Anda menggunakan *starter code* mentah (tanpa implementasi Modul 8), simulasi akan terhenti seketika dan menghasilkan pesan *error: kernel_main() kembali dari thread_block()!*.

1. Pastikan Anda sudah memiliki direktori *starter code* OS dengan file `src/thread.c` yang sudah diimplementasikan pada Modul 8 (atau modul sebelumnya). Jika Anda sudah memiliki direktori kerja dari Modul 9 atau Modul 10 yang berfungsi, Anda dapat langsung menggunakannya.

Catatan: Jika Anda belum memiliki direktori kerja, lakukan langkah berikut untuk membuat yang baru:

```
1 mkdir praktikum-minggu6
2 cd praktikum-minggu6
3 git clone https://github.com/sinavarasina/starter-code-os32.git
4 cd starter-code-os32
5
```

Kode 1.4: Membuat direktori kerja baru (opsional, jika belum punya)

2. **Langkah Kritis (hanya jika clone baru):** Jika Anda baru saja melakukan *clone* ulang repositori pada langkah sebelumnya, file `src/thread.c` masih berupa

stub kosong. Anda **harus** menyimpannya dengan implementasi `thread.c` yang sudah berfungsi dari Modul 8.

Cara Salin manual: Salin file `thread.c` yang sudah Anda kerjakan pada Modul 8 ke dalam folder `src/`.

```
1 cp /path/ke/modul8/starter-code-os32/src/thread.c src/thread.c
2
```

Kode 1.5: Menyalin `thread.c` dari direktori sebelumnya

Catatan: Jika Anda tidak menyimpan file `thread.c` dari pertemuan sebelumnya, silakan hubungi asisten praktikum.

Penting: File `thread.c` yang disalin harus mengandung baris `#include "thread.h"` di awal file. Jika baris tersebut tidak ada, tambahkan tiga baris berikut di **paling atas** file `src/thread.c`:

```
1 #include "thread.h"
2 #include "types.h"
3 #include "vga.h"
4
```

Kode 1.6: Header include yang wajib ada di awal `thread.c`

3. Verifikasi bahwa seluruh berkas sumber (*source code*) sudah lengkap.

```
1 ls src/
2 # Harus menampilkan: boot.asm  kernel.c  scheduler.h  synch.c
   synch.h
3 #
   switch.asm  thread.c  thread.h
4 #
   types.h  vga.c  vga.h
5
```

Kode 1.7: Verifikasi direktori source code

Jika ada file yang hilang, ulangi proses clone dari langkah 1.

1.4.2 Sesi 1: Menjalankan Demo Round-Robin dengan Thread Berprioritas Berbeda

Pada sesi ini, kita akan menjalankan sistem operasi dengan skenario demo yang membuat tiga *thread* berprioritas berbeda. Dari pengamatan ini, kita bisa melihat langsung bahwa *scheduler Round-Robin* tetap memperlakukan ketiga *thread* tersebut secara setara meskipun prioritasnya berbeda. **Tidak perlu mengubah kode apa pun**, cukup amati keluaran (*output*) yang muncul di layar QEMU.

Penting: Flag `PRIORITY_SCHEDULING` pada `Makefile` **hanya mengubah skenario demo** di `kernel.c`, yaitu membuat tiga *thread* dengan prioritas berbeda (Low, Med, High). Flag ini **tidak** mengubah algoritma *scheduler*. Algoritma *scheduler* tetap ditentukan oleh implementasi fungsi `next_thread_to_run()` di `thread.c`, yang saat ini masih berupa *Round-Robin* (FIFO).

Perhatikan bahwa pada file `kernel.c`, blok `#elif defined(PRIORITY_SCHEDULING)` membuat tiga *thread* dengan prioritas **berbeda**:

```
1 thread_create("Low", 10, thread_func_generic, &args_low)
    ;
2 thread_create("Med", PRI_DEFAULT, thread_func_generic, &args_med)
    ;
3 thread_create("High", 50, thread_func_generic, &args_high
    );
```

Kode 1.8: Pembuatan thread dengan prioritas berbeda di `kernel.c`

Konfigurasi argumen masing-masing *thread*:

- Low: prioritas 10, warna abu-abu (`VGA_COLOR_DARK_GRAY`), berjalan tanpa batas (`max_ticks = 0`).
- Med: prioritas 31 (`PRI_DEFAULT`), warna cyan terang (`VGA_COLOR_BRIGHT_CYAN`), berjalan tanpa batas.
- High: prioritas 50, warna putih terang (`VGA_COLOR_BRIGHT_WHITE`), berhenti setelah 5 tick (`max_ticks = 5`).

1. Aktivasi lingkungan tersetup & terisolasi.

```
1 ./run-docker.sh
2
```

Kode 1.9: Menjalankan Docker menggunakan script

Catatan: Jika muncul error permission denied, jalankan `chmod +x run-docker.sh` terlebih dahulu. Jika Docker belum berjalan, pastikan Docker Desktop atau Docker Engine sudah aktif.

2. Kompilasi OS dengan flag `PRIORITY_SCHEDULING` diaktifkan.

```
1 make clean
2 make DEMO=-DPRIORITY_SCHEDULING
3
```

Kode 1.10: Kompilasi dengan flag `PRIORITY_SCHEDULING`

PERHATIAN (Kesalahan Umum):

- Perintah `make clean` **wajib** dijalankan terlebih dahulu untuk menghapus hasil kompilasi sebelumnya. Jika langkah ini dilewati, kode lama yang sudah dikompilasi mungkin masih digunakan dan flag `PRIORITY_SCHEDULING` tidak akan terdeteksi.
- Jika muncul error `thread.c: undefined reference to 'allocate_tid'`, berarti file `thread.c` yang digunakan tidak sesuai atau belum lengkap. Pastikan file mengandung baris `#include "thread.h"` di awal, dan ulangi langkah Persiapan.

3. Jalankan kernel di QEMU dan amati keluaran (*output*) di layar.

```
1 make run
2
```

Kode 1.11: Menjalankan kernel di QEMU

Jika muncul error `qemu-system-i386: command not found`, QEMU belum terinstal pada mesin host Anda. Keluar dari Docker (ketik `exit`), instal QEMU di host, lalu jalankan `make run` dari terminal host.

4. Amati keluaran yang muncul selama kurang lebih 10–15 detik, ambil tangkapan

pan layar (*screenshot*), lalu tutup QEMU (tekan q atau Ctrl+C).

Karena `next_thread_to_run()` masih menggunakan FIFO (*Round-Robin*), ketiga *thread* bergantian **tanpa memperhatikan prioritas**:

```
1  =====
2  IF-OS | Praktikum Sistem Operasi
3  =====
4
5  Inisialisasi sistem thread...
6  Demo Minggu 6: Priority Scheduling
7  (Jika masih Round-Robin, next_thread_to_run() belum
   dimodifikasi)
8  (High=50 jalan 5 tick lalu exit, Med=31 mengambil alih, Low=10
   terakhir)
9
10 [Low] tick 0
11 [Med] tick 0
12 [High] tick 0
13 [Low] tick 1
14 [Med] tick 1
15 [High] tick 1
16 ...
17 [High] selesai (5 tick)
18 [Low] tick 5
19 [Med] tick 5
20 [Low] tick 6
21 [Med] tick 6
22 ...
23
```

Kode 1.12: Keluaran QEMU: Round-Robin mengabaikan prioritas

Catatan: Pesan header pada output QEMU mendeskripsikan perilaku yang baru akan terjadi setelah Modul 12. Pada modul ini, ketiga thread masih berjalan bergantian secara merata.

Perhatikan bahwa Low (prioritas 10) mendapat giliran yang **sama** dengan High (prioritas 50). Ketiganya bergilir secara merata: Low → Med → High

→ Low → Dalam skenario sistem nyata, ini adalah perilaku yang tidak diinginkan karena tugas berprioritas tinggi seharusnya didahulukan.

5. Bandingkan keluaran di atas dengan tabel ekspektasi berikut:

Tabel 1.2: Ekspektasi urutan eksekusi pada Round-Robin

Giliran	Thread	Prioritas	Keterangan
1	Low	10	Masuk antrean pertama kali
2	Med	31	Masuk antrean kedua
3	High	50	Masuk antrean ketiga
4	Low	10	Kembali ke depan antrean (FIFO berputar)
5	Med	31	... dan seterusnya

Urutan di atas membuktikan bahwa *scheduler Round-Robin* sepenuhnya mengabaikan kolom "Prioritas". Thread Low (10) selalu dieksekusi sebelum High (50) karena Low masuk antrean lebih dulu.

Screenshot QEMU akan ditampilkan di sini

Gambar 1.1: Keluaran QEMU menunjukkan ketiga thread berjalan bergantian secara merata meskipun prioritasnya berbeda, yang membuktikan bahwa Round-Robin mengabaikan nilai prioritas. Keterbatasan ini akan diatasi pada Modul 12.

CHAPTER 2

Penjadwalan Proses Berbasis Prioritas

2.1 Tujuan dan Output Praktikum

2.1.1 Tujuan Praktikum

Setelah menyelesaikan praktikum ini, mahasiswa diharapkan mampu:

1. Memahami konsep *Priority Scheduling* sebagai evolusi dari algoritma *Round-Robin* yang telah dianalisis pada Modul 11.
2. Memodifikasi fungsi `next_thread_to_run()` di `src/thread.c` agar memilih *thread* dengan nilai prioritas tertinggi dari *ready queue*, menggantikan logika FIFO yang ada.
3. Memverifikasi bahwa *thread* berprioritas tinggi dieksekusi lebih dahulu melalui pengujian di QEMU dengan *flag* `PRIORITY_SCHEDULING`.
4. Menganalisis fenomena *starvation* yang muncul sebagai konsekuensi dari *Priority Scheduling* murni dan memahami solusi konseptualnya.

2.1.2 Output Praktikum

Pada akhir praktikum ini, mahasiswa diharapkan menghasilkan:

- Implementasi fungsional *Priority Scheduling* pada fungsi

`next_thread_to_run()` yang terbukti mampu memilih *thread* berprioritas tertinggi dari *ready queue*.

- Tangkapan layar (*screenshot*) keluaran QEMU yang menunjukkan *thread* High (prioritas 50) dieksekusi terlebih dahulu selama 5 tick, kemudian Med (prioritas 31) mendominasi CPU sepenuhnya, sementara Low (prioritas 10) mengalami *starvation* dan tidak pernah mendapat giliran (selama Med masih ada di antrean).
- Laporan perbandingan yang menjabarkan perbedaan perilaku *Round-Robin* (Modul 11) vs *Priority Scheduling* (Modul 12) berdasarkan keluaran QEMU.

2.2 Dasar Teori

Pada Modul 11, kita sudah melihat bahwa algoritma *Round-Robin* memperlakukan semua *thread* secara setara tanpa membedakan tingkat kepentingan masing-masing tugas. Modul ini akan membahas *Priority Scheduling* sebagai alternatif yang mengatasi keterbatasan tersebut.

2.2.1 Priority Scheduling

Priority Scheduling adalah algoritma penjadwalan CPU yang menetapkan nilai prioritas numerik pada setiap *thread* [1]. Ketika *scheduler* harus memilih *thread* berikutnya untuk dieksekusi, ia akan **memindai seluruh** *ready queue* dan memilih *thread* dengan nilai prioritas tertinggi, bukan yang paling depan di antrean.

Analoginya seperti ruang gawat darurat di rumah sakit. Meskipun ada antrean pasien yang datang lebih dulu, pasien dengan kondisi paling kritis (prioritas tertinggi) akan ditangani terlebih dahulu, tanpa memperhatikan urutan kedatangan.

Pada *starter code* OS, prioritas didefinisikan melalui tiga konstanta di `src/thread.h`:

```
1 #define PRI_MIN      0      /* Prioritas terendah */
2 #define PRI_DEFAULT  31     /* Prioritas default */
3 #define PRI_MAX      63     /* Prioritas tertinggi */
```

Kode 2.1: Konstanta prioritas pada `thread.h`

Field `priority` pada `struct thread` sudah tersedia dan diinisialisasi sejak Modul 8, namun belum pernah dimanfaatkan oleh *scheduler*. Pada modul ini, field tersebut akan menjadi kunci utama dalam menentukan giliran CPU.

2.2.2 Perbedaan Mendasar dengan Round-Robin

Perbedaan utama antara *Priority Scheduling* dan *Round-Robin* terletak pada cara `next_thread_to_run()` memilih *thread* berikutnya dari antrean. Pada *Round-Robin*, pemilihan dilakukan secara berurutan berdasarkan kedatangan. Pada *Priority Scheduling*, pemilihan dilakukan berdasarkan nilai prioritas tertinggi.

- **Round-Robin:** Selalu mengambil *thread* dari posisi depan antrean (`ready_head`). Tidak ada pemindaian, tidak ada perbandingan. Kompleksitas: $O(1)$.
- **Priority Scheduling:** Memindai *seluruh* isi antrean untuk menemukan *thread* dengan *priority* tertinggi, mengeluarkannya dari posisinya (bukan dari depan), lalu menggeser elemen sisanya. Kompleksitas: $O(n)$, di mana n adalah jumlah *thread* di antrean.

Berikut ilustrasi perbedaan keduanya pada antrean yang sama:

```
1 Ready Queue: [Low(10)] -> [Med(31)] -> [High(50)]
2
3 Round-Robin memilih : Low (posisi depan, FIFO)
4 Priority memilih      : High (nilai priority tertinggi)
```

Kode 2.2: Ilustrasi perbedaan pemilihan thread

2.2.3 Starvation (Kelaparan)

Perbedaan cara distribusi CPU antara kedua algoritma ini menimbulkan konsekuensi yang cukup signifikan. Pada *Round-Robin*, setiap *thread* mendapat giliran secara merata. Pada *Priority Scheduling*, muncul risiko yang tidak ada pada *Round-Robin*, yaitu *starvation*.

Starvation adalah kondisi di mana sebuah *thread* berprioritas rendah **tidak pernah** mendapat giliran CPU karena selalu ada *thread* berprioritas lebih tinggi yang siap dieksekusi di *ready queue*.

Pada skenario praktikum ini, *starvation* total tidak terjadi karena High memiliki batas `max_ticks = 5` dan akan `exit` setelah 5 iterasi, sehingga akhirnya Med dan

Tabel 2.1: Perbandingan Round-Robin dan Priority Scheduling

Aspek	Round-Robin (Modul 11)	Priority Scheduling (Modul 12)
Prinsip pemilihan	FIFO: <i>thread</i> yang lebih dulu masuk antrian, lebih dulu dieksekusi.	<i>Thread</i> dengan nilai prioritas tertinggi selalu dipilih terlebih dahulu.
Fairness	Tinggi: setiap <i>thread</i> mendapat porsi CPU yang setara.	Rendah: <i>thread</i> berprioritas rendah bisa tidak pernah mendapat CPU.
Responsiveness	Rendah: <i>thread</i> penting harus menunggu giliran yang sama.	Tinggi: <i>thread</i> kritis langsung dieksekusi.
Kompleksitas	$O(1)$: langsung ambil dari depan antrian.	$O(n)$: harus memindai seluruh antrian.
Risiko utama	Tidak ada <i>starvation</i> , tetapi tugas penting mungkin lambat ditangani.	<i>Starvation</i> pada <i>thread</i> berprioritas rendah.

Low mendapat giliran. Namun, jika High berjalan tanpa batas, Low akan mengalami *starvation*.

Berikut skenario terjadinya *starvation* langkah demi langkah:

Tabel 2.2: Skenario starvation pada Priority Scheduling

Langkah	Thread Aktif	Yang Terjadi
1	High (pri=50)	Dipilih scheduler, berjalan, memanggil <code>yield()</code>
2	—	Scheduler memindai antrian: Low(10), Med(31), High(50)
3	High (pri=50)	Dipilih lagi karena prioritas tertinggi
4	—	High terus dipilih berulang kali
∞	Low (pri=10)	Tidak pernah mendapat giliran = <i>starvation</i>

2.2.4 Solusi: Aging

Salah satu cara untuk mengatasi *starvation* adalah dengan teknik *aging*. Ide dasarnya cukup sederhana: setiap kali *scheduler* dipanggil, prioritas semua *thread* yang **tidak** terpilih (masih menunggu di antrian) dinaikkan sebesar 1. Dengan demikian, *thread* berprioritas rendah yang sudah lama menunggu pada akhirnya akan memiliki prioritas yang cukup tinggi untuk mendapat giliran.

```

1 function next_thread_to_run():
2     chosen = thread dengan prioritas tertinggi

```

```

3
4     for setiap thread LAIN di ready_queue:
5         thread.priority += 1  /* Naikkan 1 setiap kali tidak
           terpilih */
6
7     return chosen

```

Kode 2.3: Pseudocode aging

Tabel 2.3 merangkum konsep-konsep penting pada modul ini.

Tabel 2.3: Tabel Ringkasan Dasar Teori Modul 12

Konsep	Penjelasan Singkat
Priority Scheduling	Algoritma penjadwalan yang memilih <i>thread</i> berdasarkan nilai prioritas tertinggi di <i>ready queue</i> .
Starvation	Kondisi di mana <i>thread</i> berprioritas rendah tidak pernah mendapat giliran CPU karena selalu dikalahkan oleh <i>thread</i> berprioritas lebih tinggi.
Aging	Teknik pencegahan <i>starvation</i> dengan menaikkan prioritas <i>thread</i> yang sudah lama menunggu.
Pemindaian Antrean	Proses iterasi seluruh isi <i>ready queue</i> untuk menemukan <i>thread</i> dengan priority tertinggi; kompleksitas $O(n)$.

2.3 Alat dan Bahan

2.3.1 Perangkat Keras

- PC atau Laptop dengan arsitektur prosesor basis *host* x86/x64 yang mendukung kelanjutan kompilasi emulator.
- Memori minimal 4GB RAM ruang eksekusi.

2.3.2 Perangkat Lunak (Bahan Praktikum)

- OS *Host* utama (Windows dengan WSL/GitBash, Linux, atau macOS).
- Text Editor (misalnya Visual Studio Code) untuk menyunting berkas kode bahasa C dan *Assembly*.
- Instalasi Docker Engine (docker.io) untuk menyalakan lingkungan isolasi kompilasi.
- Emulator Hardware x86: `qemu-system-x86` (atau instalasi QEMU setara) yang akan mengeksekusi binari hasil OS.
- Direktori *Starter Code* OS 32-bit dengan implementasi Modul 8 yang sudah berfungsi (berisi direktori `src` dengan kerangka lengkap `kernel.c`, `thread.h`, `thread.c`, `switch.asm`, `dsb`).

2.4 Langkah-Langkah Praktikum

Pada bagian ini, kita akan memodifikasi fungsi `next_thread_to_run()` agar menggunakan *Priority Scheduling* alih-alih *Round-Robin*. Setelah modifikasi selesai, kita akan menjalankan sistem di QEMU dan mengamati langsung fenomena *starvation* yang terjadi.

2.4.1 Persiapan Lingkungan

Pada praktikum ini, Anda akan memodifikasi fungsi `next_thread_to_run()` di `src/thread.c` agar mengimplementasikan *Priority Scheduling*. File yang dimodifikasi hanya `src/thread.c`. Pastikan Anda menggunakan direktori kerja yang sama dengan Modul 11.

Catatan: Pastikan Anda masih berada di dalam lingkungan Docker yang digunakan pada Modul 11, atau jalankan kembali `./run-docker.sh` jika sudah keluar.

1. Buka terminal, pastikan Anda berada di dalam direktori *source code* OS (direktori kerja yang sama dengan Modul 11).
2. Bersihkan hasil *build* sebelumnya agar kompilasi berikutnya menggunakan kode terbaru yang akan Anda tulis.

```
1 make clean
2
```

Kode 2.4: Bersihkan build sebelumnya

2.4.2 Sesi 1: Analisis Kode Scheduler Round-Robin yang Akan Dimodifikasi

Sebelum menulis implementasi *Priority Scheduling*, kita perlu memahami terlebih dahulu bagaimana kode *Round-Robin* yang ada saat ini bekerja. Dengan memahami struktur yang sudah ada, kita bisa mengidentifikasi bagian mana yang perlu

diubah. Buka file `src/thread.c` menggunakan teks editor Anda dan cari fungsi `next_thread_to_run()`.

1. Perhatikan implementasi *Round-Robin* (FIFO) yang Anda tulis pada Modul 8:

```
1 struct thread *next_thread_to_run(void) {
2     if (ready_count == 0)
3         return initial_thread;
4     struct thread *next = ready_queue[ready_head];
5     ready_head = (ready_head + 1) % (MAX_THREADS * 2);
6     ready_count--;
7     return next;
8 }
9
```

Kode 2.5: Implementasi Round-Robin saat ini di `next_thread_to_run()`

2. Identifikasi kelemahan kode di atas: baris `ready_queue[ready_head]` selalu mengambil *thread* dari posisi depan antrean tanpa memeriksa field *priority*. Akibatnya, *thread* berprioritas tinggi diperlakukan sama dengan *thread* berprioritas rendah, sebagaimana telah dibuktikan pada keluaran QEMU di Modul 11.
3. Perhatikan juga variabel-variabel antrean yang akan Anda gunakan kembali:

```
1 static struct thread *ready_queue[MAX_THREADS * 2];
2 static uint32_t ready_head = 0;
3 static uint32_t ready_tail = 0;
4 static uint32_t ready_count = 0;
5
```

Kode 2.6: Variabel antrean thread di `thread.c` (sudah disediakan)

Antrean ini berupa *circular array* dengan kapasitas `MAX_THREADS * 2` (16 slot). Variabel `ready_head` menunjuk ke elemen terdepan, `ready_tail` menunjuk ke posisi kosong berikutnya, dan `ready_count` menghitung jumlah *thread* yang saat ini ada di antrean.

2.4.3 Sesi 2: Implementasi Priority Scheduling

Setelah melihat kelemahan pada implementasi *Round-Robin*, kita bisa mulai memodifikasi fungsi `next_thread_to_run()`. Logika baru yang akan kita tulis terdiri dari tiga tahap: pertama, memindai antrean untuk menemukan *thread* berprioritas tertinggi. Kedua, mengambil *thread* tersebut dari antrean. Ketiga, menggeser elemen antrean untuk menutup celah yang ditinggalkan. Masih di dalam file `src/thread.c`:

1. Implementasi `next_thread_to_run()` (Priority Scheduling)

Ganti seluruh isi fungsi `next_thread_to_run()` dengan kode berikut. Logikanya terdiri dari tiga tahap yang ditandai dengan komentar bernomor:

```
1 struct thread *next_thread_to_run(void) {
2     if (ready_count == 0)
3         return initial_thread;
4
5     /* 1. Cari indeks thread dengan prioritas tertinggi */
6     uint32_t best_idx = ready_head;
7     int      best_prio = ready_queue[ready_head]->priority;
8
9     for (uint32_t i = 1; i < ready_count; i++) {
10         uint32_t idx = (ready_head + i) % (MAX_THREADS * 2);
11         if (ready_queue[idx]->priority > best_prio) {
12             best_prio = ready_queue[idx]->priority;
13             best_idx = idx;
14         }
15     }
16
17     /* 2. Ambil thread terpilih dari antrean */
18     struct thread *next = ready_queue[best_idx];
19
20     /* 3. Geser elemen setelah posisi terpilih untuk menutup
21     celah */
22     uint32_t i = best_idx;
23     while (1) {
24         uint32_t next_i = (i + 1) % (MAX_THREADS * 2);
```

```
24     if (next_i == ready_tail) break;
25     ready_queue[i] = ready_queue[next_i];
26     i = next_i;
27 }
28 ready_tail = (ready_tail == 0)
29             ? (MAX_THREADS * 2 - 1)
30             : ready_tail - 1;
31 ready_count--;
32
33 return next;
34 }
35
```

Kode 2.7: Kode implementasi Priority Scheduling pada next_thread_to_run()

Penjelasan setiap tahap:

- **Tahap 1 (Pemindaian):** Loop menelusuri seluruh isi antrian mulai dari ready_head. Untuk setiap elemen, bandingkan priority-nya dengan best_prio. Jika lebih tinggi, catat indeks aktualnya di best_idx dan perbarui best_prio. Setelah loop selesai, best_idx menunjuk ke posisi thread dengan prioritas tertinggi.
- **Tahap 2 (Pengambilan):** Simpan pointer thread terpilih dari ready_queue[best_idx] ke variabel next.
- **Tahap 3 (Penggeseeran):** Karena thread yang diambil mungkin bukan dari posisi depan antrian (berbeda dengan FIFO), elemen-elemen setelahnya harus digeser maju satu posisi untuk menutup celah. Ini dilakukan oleh loop while yang bergerak dari best_idx hingga ready_tail. Terakhir, mundurkan ready_tail dan kurangi ready_count.

Catatan penting: Jika muncul error implicit declaration of function saat kompilasi, pastikan Anda tidak menghapus baris #include "thread.h" dan #include "types.h" di awal file.

Jika muncul warning comparison of unsigned expression < 0 is always true, pastikan tipe data ready_count adalah uint32_t (bukan int). Cek deklarasi variabel di bagian atas file thread.c.

2. **Simpan** file `thread.c`. Saat ini *scheduler* Anda sudah berubah dari *Round-Robin* menjadi *Priority Scheduling*.

2.4.4 Sesi 3: Kompilasi dan Verifikasi Sistem OS

Setelah kode *Priority Scheduling* selesai ditulis, kita bisa mengompilasi dan menjalankan sistem untuk melihat hasilnya. Pada sesi ini, kita akan membandingkan keluaran QEMU sebelum dan sesudah modifikasi.

1. Lakukan kompilasi sistem operasi Anda dengan *flag* `PRIORITY_SCHEDULING`.

```
1 # Pastikan telah menjalankan make clean sebelumnya
2 make DEMO=-DPRIORITY_SCHEDULING
3 make run
4
```

Kode 2.8: Kompilasi dan jalankan dengan Priority Scheduling

Jika muncul error linker error: undefined reference to 'next_thread_to_run', berarti ada kesalahan sintaks pada fungsi Anda (misalnya kurung kurawal tidak tertutup). Periksa kembali kode pada Sesi 2. Jika kompilasi berhasil tetapi QEMU langsung menampilkan KERNEL PANIC, kemungkinan `next_thread_to_run()` mengembalikan NULL. Pastikan blok `if (ready_count == 0) return initial_thread;` masih ada di awal fungsi.

2. **Verifikasi Hasil:** Amati keluaran (*output*) di layar QEMU. Bandingkan dengan perilaku sebelum modifikasi.
 - **Sebelumnya (Modul 11):** Ketiga *thread* bergantian secara merata tanpa memperhatikan prioritas (*Round-Robin*).
 - **Sekarang (Modul 12):** Thread High (prioritas 50) mendominasi CPU selama 5 tick penuh. Setelah High *exit*, Med (prioritas 31) mendominasi CPU sepenuhnya, sementara Low (prioritas 10) **tidak pernah mendapat giliran** (*starvation*).

```
1 =====
2 IF-OS | Praktikum Sistem Operasi
```

```

3 =====
4
5 Inisialisasi sistem thread...
6 Demo Minggu 6: Priority Scheduling
7 (Jika masih Round-Robin, next_thread_to_run() belum
   dimodifikasi)
8 (High=50 jalan 5 tick lalu exit, Med=31 mengambil alih, Low=10
   terakhir)
9
10 [High] tick 0
11 [High] tick 1
12 [High] tick 2
13 [High] tick 3
14 [High] tick 4
15 [High] selesai (5 tick)
16 [Med] tick 0
17 [Med] tick 1
18 [Med] tick 2
19 [Med] tick 3
20 ...           <- Med berjalan terus, Low tidak pernah muncul
21

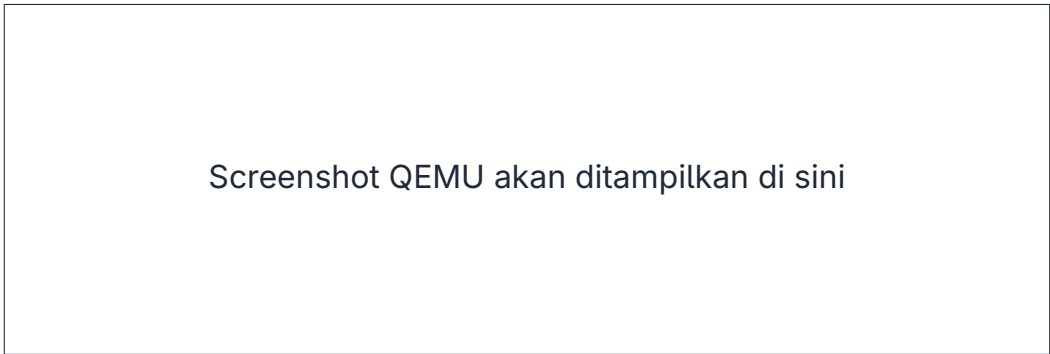
```

Kode 2.9: Ekspektasi keluaran QEMU setelah Priority Scheduling

Meskipun *thread* Low memanggil `yield()` dan masuk kembali ke antrian, *scheduler* selalu memilih *thread* berprioritas tertinggi. Karena Med (31) selalu memiliki prioritas lebih tinggi daripada Low (10), Med akan terus dipilih dan Low **tidak pernah mendapat giliran sama sekali**.

Fenomena ini adalah contoh nyata dari **starvation** yang telah dibahas pada Dasar Teori. Pada skenario ini, High tidak menyebabkan *starvation* permanen karena memiliki batas `max_ticks = 5`, tetapi Med (yang berjalan tanpa batas) secara efektif membuat Low kelaparan selamanya. Untuk mengatasi ini, diperlukan mekanisme *aging* sebagaimana dijelaskan pada Dasar Teori, namun implementasinya berada di luar cakupan modul ini.

3. Ambil tangkapan layar (*screenshot*) hasil eksekusi untuk membuktikan bahwa implementasi *Priority Scheduling* telah berjalan sesuai ekspektasi.



Screenshot QEMU akan ditampilkan di sini

Gambar 2.1: Keluaran (*output*) QEMU setelah implementasi Priority Scheduling. Thread High (prioritas 50) dieksekusi pertama kali selama 5 tick penuh, kemudian Med (prioritas 31) mendominasi CPU sepenuhnya sementara Low (prioritas 10) mengalami *starvation*.

Bibliography

- [1] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Wiley, 2018.